ClaimsGuard AI Design

Generated: 2026-10-31T06:07:46.999Z • Source: Shared ChatGPT link

USER

7. Claims Leakage Detection Agent
Category: AI for Insurance Claims Intelligence & Financial Leakage Prevention
Applicable Domains: General Insurance, Health Insurance, Motor Insurance and Claims Management
Focus: Claims Validation, Duplicate Detection, Overpayment Prevention, Policy Compliance, Investigation and Settlement Review
Problem Statement
Insurance companies process large volumes of claims involving policy coverage, supporting documents, invoices, service providers, claim histories and settlement calculations. Each claim must be checked against the applicable policy terms before an amount is approved for payment.
Financial leakage occurs when claims are paid incorrectly. This may result from duplicate submissions, excessive charges, incorrect calculations, missed policy exclusions, inappropriate benefits or simple processing errors. Individually, these amounts may appear small, but across a large claims portfolio they can represent a significant and avoidable financial loss.
Not every leakage event is fraudulent. Many result from operational mistakes, incomplete information or inconsistent application of policy rules. A claims handler under time pressure may overlook a deductible, approve an invoice line that was already paid under an earlier claim or miss that a treatment falls under a policy exclusion.
For example, a health insurance claim may include a hospital invoice in which the same diagnostic test appears twice, a room charge that exceeds the policy's daily limit and a consumable item that the policy explicitly excludes. Each issue may be minor on its own, but together they could cause the insurer to overpay by a material amount.
The challenge is to build an AI-powered Claims Leakage Detection Agent that examines insurance claims, identifies potential financial leakage, explains the supporting evidence and enables claims teams to investigate and correct questionable outcomes before settlement. The solution should combine structured claims data, policy conditions, supporting documents and historical claim information.
The application must distinguish confirmed rule violations from suspicious patterns requiring human review. It should calculate the potential leakage amount wherever it can be determined, explain why a claim was flagged, retain the supporting evidence and allow authorised users to confirm, correct or dismiss each finding.
Participants are encouraged to introduce additional leakage detection techniques and innovative approaches beyond the specified scenarios. Solutions demonstrating meaningful detection of complex or emerging leakage patterns will receive additional consideration within the evaluation framework.
The organiser will assess actual application behaviour rather than a static dashboard, predefined presentation or standalone AI-generated response.
Claims Leakage Scenarios to Be Addressed
The solution should consider the following types of claims leakage. These scenarios represent

practical challenges faced by insurers and provide the basis for progressive evaluation.

1. Duplicate Claims and Invoice Line Items

The same incident, invoice or charge may be submitted more than once, either within a single claim or across separate claims. The application should identify exact duplicates as well as near-duplicates with minor differences in dates, amounts or reference numbers.

2. Policy Limit Breaches and Excluded Services

A claim may request an amount that exceeds an applicable coverage limit or include expenses that are not covered under the policy. The system should evaluate each claim line against the relevant policy terms and identify the specific limit or exclusion involved.

3. Incorrect Settlement Calculation

Deductibles, co-payments, sub-limits and benefit caps may be applied incorrectly. The application should recalculate the expected settlement amount using the supplied policy rules and highlight any differences from the proposed settlement.

4. Unusual Provider Billing and Related-Party Claims

A provider's charges may be materially inconsistent with comparable claims, or several claims may share unusual combinations of claimant, provider, invoice or incident information. The system should identify these patterns for review without automatically concluding that they are fraudulent.

5. Legitimate Lookalikes and Incomplete Evidence

Similar claims may represent separate valid incidents, such as repeated physiotherapy sessions, and must not automatically be rejected. Equally, some claims cannot be assessed reliably because essential information is missing. The application should request clarification rather than inventing a finding.

Level 1: Validate Claims and Detect Potential Leakage

Suggested time: 0 - 3 hours

Objective: Build a working claims validation application that ingests claims and policy information, detects common leakage patterns and presents explainable alerts for review.

Required Capabilities

1. Claims Data Ingestion

Build an application that allows users to upload or select synthetic claims information. The application should support the following sample data:

• Claim records containing claim identifiers, policy numbers, claimants, incident dates, claim types and requested amounts.

• Invoice line items containing service descriptions, codes, quantities, unit prices, providers and invoice references.

• Policy details containing coverage, exclusions, deductibles, co-payments, sub-limits and benefit periods.

• Claim histories and settlement records containing previously approved and paid amounts.

The application should accept CSV, Excel, JSON and sample document uploads. It must validate identifiers, amounts, dates and mandatory fields, and must avoid silently ignoring invalid records.

2. Policy Rule Validation

Implement a validation mechanism that evaluates each claim and claim line against the applicable policy. At a minimum, the application must check coverage status, exclusions, deductibles, co-payments, sub-limits and overall benefit limits. Each validation result should

identify the specific policy rule applied and the outcome.
3. Duplicate Detection
Identify exact and potential duplicates using incident references, invoice numbers, dates, amounts, providers and other relevant attributes. At a minimum, the application must handle the following scenarios:
Scenario Expected detection behaviour
Exact duplicate claim Identify the same invoice or incident submitted more than once
Duplicate line item Identify the same charge repeated within or across claims
Near-duplicate Flag claims with minor differences in dates, amounts or references for review
Legitimate repeat service Avoid flagging separate valid services merely because they appear similar
4. Leakage Assessment
For every detected issue, calculate the potential excess amount where it can be determined. For example, if a room charge exceeds the policy's daily limit by 2,000 for three days, the calculated discrepancy is 6,000. The application must clearly distinguish a calculated discrepancy from an estimated or suspected leakage amount and explain the calculation.
5. Claims Alert Dashboard
Develop a dashboard where users can inspect flagged claims. The dashboard should display the number of claims processed, flagged claims, leakage categories, potential amounts and severity. Users should be able to filter by claim type, provider, leakage category, amount and date. Selecting an alert should display the underlying claim, the rule or pattern that triggered it and the supporting records.
Expected Submission
A working claims validation application demonstrating data ingestion, policy checks, duplicate detection, leakage calculation and an explainable alert dashboard.

ASSISTANT

This is a strong use case for an **AI-powered insurance claims auditing and leakage prevention platform**. The key is to build an application that actually processes claims, applies policy rules, detects anomalies, calculates excess payments and supports human investigation.

I would approach this as an end-to-end application rather than an AI chatbot with a dashboard.

1. What we need to build

2. System architecture

I recommend a hybrid architecture combining deterministic rules, machine learning and LLM-based document intelligence.

Layer	Technology	Responsibility
Frontend	React + Tailwind	Dashboard and investigation
Backend	Python + FastAPI	API and orchestration

Layer	Technology	Responsibility
Database	PostgreSQL	Claims, policies, audit logs
Data processing	Pandas	Claims processing
Duplicate detection	RapidFuzz	Fuzzy matching
ML	Scikit-learn	Anomaly detection
Document AI	OCR + LLM	Invoice and policy extraction
Vector search	FAISS	Semantic policy retrieval
Background jobs	Celery + Redis	Batch processing

Important: Do not use an LLM to calculate settlement amounts. Use deterministic Python calculations. LLMs should extract information and explain findings.

3. Core detection engine

These are the detection mechanisms I would implement.

Detection category	Technique	Example
Exact duplicates	Hash matching	Same invoice submitted twice
Near duplicates	Fuzzy matching	Invoice number INV102 vs INV-102
Duplicate line items	Composite keys	Same diagnostic test billed twice
Coverage violations	Rule engine	Excluded medical consumable
Room rent breach	Policy calculation	7,000 charged, 5,000 limit
Deductible error	Settlement calculation	10,000 deductible missed
Co-payment error	Percentage calculation	20% co-pay not deducted
Provider anomalies	Statistical analysis	Provider charges 3x peer average
Related-party patterns	Graph analysis	Multiple claims sharing unusual attributes
Missing evidence	Document validation	Missing invoice or discharge summary

4. Leakage calculation engine

This is one of the most important components.

Example:

Health insurance claim

Item	Requested	Policy allowed	Leakage
Room rent (3 days)	21,000	15,000	6,000
Diagnostic test duplicate	4,000	2,000	2,000
Consumables excluded	3,000	0	3,000
Total	**28,000**	**17,000**	**11,000**

Then apply deductible and co-payment rules to calculate the correct settlement.

The system should distinguish:

- **Confirmed discrepancy:** Directly established by policy or invoice evidence.
- **Potential leakage:** Suspicious pattern that requires investigation.
- **Unassessed:** Insufficient information to determine the correct amount.

Do not add suspected amounts to confirmed savings.

5. Agent workflow

I would implement the following pipeline:

6. Dashboard design

I would build five primary screens.

7. Database design

Recommended PostgreSQL entities:

Table	Purpose
claims	Main claim records
policies	Policy information
policy_rules	Limits, exclusions, deductibles
claim_items	Individual invoice line items
invoices	Invoice metadata
providers	Hospital, garage and service providers

Table	Purpose
claim_documents	Uploaded documents
settlements	Proposed and actual settlements
leakage_findings	Detected issues
investigations	Reviewer decisions
audit_logs	Complete action history

8. Dataset requirements

For a convincing demonstration, create synthetic datasets covering three insurance domains.

Dataset	Records target	Important scenarios
Health insurance	1,000 claims	Room rent, exclusions, duplicate tests
Motor insurance	1,000 claims	Duplicate repairs, inflated parts, labour
General insurance	1,000 claims	Property damage, coverage limits, duplicate invoices
Policies	500	Limits, deductibles, exclusions
Providers	200	Hospitals, garages, service providers
Historical settlements	2,000	Previous payouts and claim histories

Include both clean and problematic records. Also include legitimate lookalikes and incomplete claims to test false-positive handling.

9. Development priorities (0–3 hours)

Don't attempt to build every feature at once.

Time	Deliverable	Priority
0–30 min	Dataset generation and ingestion	Critical
30–60 min	Policy validation engine	Critical
60–90 min	Duplicate detection engine	Critical
90–120 min	Leakage calculation and risk scoring	Critical
120–150 min	Investigation dashboard	Critical

Time	Deliverable	Priority
150–180 min	Testing, evidence and audit trail	Critical

10. What will make this solution stand out?

These are the differentiators I would prioritize beyond basic rule matching.

1. **Explainable findings:** Every alert must show the exact policy clause, invoice line and calculation behind it.
2. **False-positive intelligence:** Recognize legitimate repeated treatments and separate incidents.
3. **Cross-claim detection:** Identify suspicious relationships across multiple claims, not just within one claim.
4. **Provider benchmarking:** Compare billing against similar providers and procedures.
5. **Human feedback loop:** Use confirmed and dismissed findings to improve future anomaly detection.
6. **Counterfactual settlement:** Show what the insurer should have paid versus what was requested or proposed.

My recommended implementation stack

Given the three-hour evaluation window, I would use:

- **React + Tailwind:** Frontend.
- **FastAPI:** Backend.
- **PostgreSQL:** Database.
- **Pandas + RapidFuzz:** Validation and duplicate detection.
- **Scikit-learn Isolation Forest:** Anomaly detection.
- **Python rule engine:** Policy and settlement calculations.
- **LLM:** Evidence summarization and policy clause explanation.

My recommendation: Start with Health Insurance as the primary working demonstration. It offers clear, measurable leakage scenarios such as duplicate diagnostics, room rent limits, excluded consumables, deductibles and co-payments. Keep the architecture domain-independent so Motor and General Insurance can be added without rewriting the detection engine.

The most important evaluation criterion is that uploading a dataset must trigger actual processing and produce traceable findings—not merely populate a dashboard with predefined numbers.

USER

which dataset to use which model to train? from datasets import load_dataset

ds = load_dataset("ziadatalabs/FreeInsuranceClaims100M") thinking to go with this what insights to note as part of research can u give in deliverables

ASSISTANT

1. Dataset assessment: FreeInsuranceClaims100M

Good choice for initial experimentation Not sufficient as the only dataset

I would use it, but with a specific role in your project: **claim-level fraud and anomaly detection**, not complete claims leakage detection.

Dataset:

What the dataset contains

Attribute	Details
Records	100 million
Format	Parquet
Data type	Synthetic structured data
Insurance types	Auto, home, renters, property, business
Target label	is_fraud_flagged_ground_truth
File size	Approximately 2.52 GB
License	CC BY-NC 4.0

It contains claim amount, deductible, policy tenure, previous claims, claim status, incident dates and fraud labels. <Cite refs={["turn377701search0","turn377701search1"]}/>

Important limitations

Your problem statement requires much more than fraud detection.

Required capability	Available in dataset?
Claim-level anomaly detection	Yes
Fraud classification	Yes
Previous claim analysis	Yes
Claim amount analysis	Yes
Deductible information	Yes
Invoice line-item duplication	No

Required capability	Available in dataset?
Policy exclusions	No
Room rent / benefit sub-limits	No
Co-payment calculations	No
Actual settlement comparison	No
Invoice documents	No
Provider billing comparison	No

There is another important issue: the dataset's fraud label is generated using patterns such as short policy tenure and unusually high claim amounts. Claim status is also correlated with the fraud label. This creates a risk of data leakage if you use claim_status as a model feature. <Cite refs= {["turn377701search0","turn377701search3"]}/>

My verdict: Keep this dataset. Do not build the entire application around it.

Also, it covers US property and casualty insurance, not Indian health insurance. If your demonstration uses INR and Indian hospital claims, create a separate synthetic health-claims dataset.

2. Which datasets should you combine?

I recommend a multi-dataset approach.

ASSISTANT

I would add these sources to your research:

Dataset	Purpose	Recommendation
	General and motor claim anomalies	Primary
	Motor fraud classification	Secondary
	Health insurance claims and payment analysis	Secondary
	Evidence and investigation scenarios	Supporting
	Human review and explainability	Supporting

The AHRQ dataset is particularly relevant because it includes inpatient, outpatient and prescription claims, payments and enrollment information. <Cite refs= {["turn659902search9","turn659902search8","turn659902search3","turn659902search1"]}/>

One limitation: these datasets cannot simply be joined together. They have different schemas, jurisdictions and data-generation processes. Use them independently for research and build a normalized synthetic dataset for your actual application.

3. Which ML models should you train?

Don't train one giant model to detect everything.

Build a multi-engine detection system.

Engine	Model / Technique	Purpose	Train?
Fraud prediction	CatBoost	Predict suspicious claims	Yes
Anomaly detection	Isolation Forest	Find unusual claims	Yes
Billing anomaly	LightGBM	Detect unusual provider charges	Yes, if labels exist
Duplicate detection	RapidFuzz + hashing	Detect duplicate claims and invoices	No
Policy validation	Python rule engine	Detect policy violations	No
Document intelligence	LLM + OCR	Extract invoice and policy information	No
Explainability	SHAP	Explain ML predictions	No

My primary model recommendation: CatBoost

Why CatBoost?

- Handles categorical variables such as claim type and state.
- Handles numerical variables such as claim amount and policy tenure.
- Performs well on structured tabular datasets.
- Supports class weighting for imbalanced data.
- Can be explained using SHAP.

Suggested training configuration

Parameter	Starting value
Model	CatBoostClassifier
Iterations	800

Parameter	Starting value
Depth	8
Learning rate	0.05
Loss	Logloss
Evaluation	PR-AUC
Class imbalance	Auto class weights
Validation	Temporal split

These are initial hyperparameters, not guaranteed optimal settings.

Important feature selection:

Use:

- claim_type
- claim_amount
- deductible
- policyholder_tenure_years
- previous_claims_count
- claim_to_type_median_ratio
- days_between_incident_and_filing

Exclude:

- claim_status
- days_to_resolution

These can reveal the outcome of the claim and create target leakage.

Also, because the ground-truth fraud label is synthetically generated using claim tenure and amount patterns, report how much of your model's performance comes from learning those artificial patterns.

Second model: Isolation Forest

Use it to identify:

- Unusually high claim amounts.
- Unusual claim frequency.
- Unusual provider billing.
- Abnormal combinations of claim characteristics.

This is useful because not every leakage event has a fraud label.

ASSISTANT

1. Dataset assessment: FreeInsuranceClaims100M

Good choice for initial experimentation Not sufficient as the only dataset

I would use it, but with a specific role in your project: **claim-level fraud and anomaly detection**, not complete claims leakage detection.

Dataset:

What the dataset contains

Attribute	Details
Records	100 million
Format	Parquet
Data type	Synthetic structured data
Insurance types	Auto, home, renters, property, business
Target label	is_fraud_flagged_ground_truth
File size	Approximately 2.52 GB
License	CC BY-NC 4.0

It contains claim amount, deductible, policy tenure, previous claims, claim status, incident dates and fraud labels. <Cite refs={["turn377701search0","turn377701search1"]}/>

Important limitations

Your problem statement requires much more than fraud detection.

Required capability	Available in dataset?
Claim-level anomaly detection	Yes
Fraud classification	Yes
Previous claim analysis	Yes
Claim amount analysis	Yes
Deductible information	Yes
Invoice line-item duplication	No

Required capability	Available in dataset?
Policy exclusions	No
Room rent / benefit sub-limits	No
Co-payment calculations	No
Actual settlement comparison	No
Invoice documents	No
Provider billing comparison	No

There is another important issue: the dataset's fraud label is generated using patterns such as short policy tenure and unusually high claim amounts. Claim status is also correlated with the fraud label. This creates a risk of data leakage if you use claim_status as a model feature. <Cite refs= {["turn377701search0","turn377701search3"]}/>

My verdict: Keep this dataset. Do not build the entire application around it.

Also, it covers US property and casualty insurance, not Indian health insurance. If your demonstration uses INR and Indian hospital claims, create a separate synthetic health-claims dataset.

---

2. Which datasets should you combine?

I recommend a multi-dataset approach.

I would add these sources to your research:

Dataset	Purpose	Recommendation
	General and motor claim anomalies	Primary
	Motor fraud classification	Secondary
	Health insurance claims and payment analysis	Secondary
	Evidence and investigation scenarios	Supporting
	Human review and explainability	Supporting

The AHRQ dataset is particularly relevant because it includes inpatient, outpatient and prescription claims, payments and enrollment information. <Cite refs= {["turn659902search9","turn659902search8","turn659902search3","turn659902search1"]}/>

One limitation: these datasets cannot simply be joined together. They have different schemas, jurisdictions and data-generation processes. Use them independently for research and build a normalized synthetic dataset for your actual application.

3. Which ML models should you train?

Don't train one giant model to detect everything.

Build a multi-engine detection system.

Engine	Model / Technique	Purpose	Train?
Fraud prediction	CatBoost	Predict suspicious claims	Yes
Anomaly detection	Isolation Forest	Find unusual claims	Yes
Billing anomaly	LightGBM	Detect unusual provider charges	Yes, if labels exist
Duplicate detection	RapidFuzz + hashing	Detect duplicate claims and invoices	No
Policy validation	Python rule engine	Detect policy violations	No
Document intelligence	LLM + OCR	Extract invoice and policy information	No
Explainability	SHAP	Explain ML predictions	No

My primary model recommendation: CatBoost

Why CatBoost?

- Handles categorical variables such as claim type and state.
- Handles numerical variables such as claim amount and policy tenure.
- Performs well on structured tabular datasets.
- Supports class weighting for imbalanced data.
- Can be explained using SHAP.

Suggested training configuration

Parameter	Starting value
Model	CatBoostClassifier
Iterations	800
Depth	8
Learning rate	0.05

Parameter	Starting value
Loss	Logloss
Evaluation	PR-AUC
Class imbalance	Auto class weights
Validation	Temporal split

These are initial hyperparameters, not guaranteed optimal settings.

Important feature selection:

Use:

- claim_type
- claim_amount
- deductible
- policyholder_tenure_years
- previous_claims_count
- claim_to_type_median_ratio
- days_between_incident_and_filing

Exclude:

- claim_status
- days_to_resolution

These can reveal the outcome of the claim and create target leakage.

Also, because the ground-truth fraud label is synthetically generated using claim tenure and amount patterns, report how much of your model's performance comes from learning those artificial patterns.

Second model: Isolation Forest

Use it to identify:

- Unusually high claim amounts.
- Unusual claim frequency.
- Unusual provider billing.
- Abnormal combinations of claim characteristics.

This is useful because not every leakage event has a fraud label.

---

4. Research insights to document

This is important for your submission. You should demonstrate that you understand the problem, not just the technology.

I recommend preparing these research findings.

Insight 1: Fraud detection is not the same as leakage detection

Fraud is intentional deception. Leakage also includes legitimate operational mistakes.

Examples:

- Duplicate invoice accidentally paid twice.
- Incorrect deductible calculation.
- Excluded expense approved by mistake.
- Provider billing exceeds contractual limits.

Research conclusion: A fraud classifier alone cannot solve claims leakage.

Insight 2: Rule-based systems are essential

Some leakage scenarios are deterministic.

For example:

$$\text{Excess} = \max(0, \text{Charged} - \text{Allowed})$$

A machine learning model should not determine whether a 5,000 policy limit was exceeded when the policy explicitly specifies that limit.

Use deterministic calculations for policy compliance.

Insight 3: Class imbalance

Most insurance claims are legitimate.

A model trained on imbalanced data can achieve high accuracy while missing suspicious claims.

Research:

- Fraud-to-legitimate claim ratio.
- Precision and recall.
- PR-AUC.
- False positive rate.
- Cost of missed leakage.

A 99% accuracy model is not necessarily useful.

Insight 4: Cross-claim relationships

A single claim may appear legitimate, but relationships between claims can reveal suspicious patterns.

Example:

- Same invoice submitted under different claim IDs.
- Same provider submitting unusually similar invoices.
- Same incident referenced across multiple claims.
- Multiple claims sharing unusual claimant-provider combinations.

Research graph-based anomaly detection as a potential extension. Research literature also highlights graph-based methods and cost-sensitive classification as promising directions. <Cite refs= {["turn659902search5","turn659902search11"]}/>

Insight 5: Explainability is necessary

Every alert should answer:

1. What was detected?
2. Which rule or model detected it?
3. What evidence supports the finding?
4. How much potential leakage is involved?
5. What should the claims handler investigate?

Insight 6: Missing information should not become a negative finding

If a policy document is missing, the application should report:

INSUFFICIENT_EVIDENCE

It should not assume the claim violates the policy.

---

5. Research deliverables

I would organize your research into these eight deliverables.

- Dataset sources and licenses.
- Dataset schema and field descriptions.
- Missing values and duplicate records.
- Class distribution.
- Fraud vs legitimate claim distribution.
- Feature correlation.
- Dataset limitations and biases.

02. Exploratory Data Analysis (EDA)

- Claim distribution by insurance type.
- Average claim amounts.
- Claim frequency by policyholder tenure.
- Claim amount vs deductible.
- Historical claim frequency.

- Suspicious claim distribution.
- Provider-level billing patterns.

Compare:

Model	Evaluation
Logistic Regression	Baseline
Random Forest	Classification
XGBoost	Classification
CatBoost	Primary model
Isolation Forest	Anomaly detection

Compare precision, recall, F1, PR-AUC, inference latency and training time.

Document:

- Coverage rules.
- Exclusions.
- Deductibles.
- Co-payment.
- Sub-limits.
- Overall benefit limits.
- Expected settlement calculations.

- Exact matching.
- Fuzzy matching.
- Invoice normalization.
- Date and amount similarity.
- Legitimate repeat service identification.

- SHAP feature importance.
- Individual claim explanations.
- Evidence-to-alert mapping.
- Confidence and uncertainty levels.
- Human review workflow.

- Total requested claim amount.
- Calculated excess amount.
- Potential leakage.
- Confirmed leakage.
- Leakage by category.
- Leakage by provider.

- False-positive investigation rate.
- Precision and recall.
- PR-AUC and F1.
- Confusion matrix.
- False-positive analysis.
- Processing throughput.
- Detection latency.
- Model limitations.

6. Dataset exploration code

For 100 million rows, don't load everything into pandas. Start with a streaming sample and use DuckDB or Polars for larger-scale analysis.

```
from datasets import load_dataset
from itertools import islice
import pandas as pd

DATASET = "ziadatalabs/FreeInsuranceClaims100M"

# Stream the dataset instead of loading all 100M records
ds = load_dataset(
    DATASET,
    split="train",
    streaming=True
)

# Take an initial 100,000-row research sample
records = list(islice(ds, 100_000))

df = pd.DataFrame(records)

print("Shape:", df.shape)
print("\nColumns:\n", df.columns.tolist())

print("\nMissing values:\n", df.isnull().sum())

print("\nFraud distribution:")
print(df["is_fraud_flagged_ground_truth"].value_counts(normalize=True))

print("\nClaim distribution:")
print(df["claim_type"].value_counts())

print("\nFraud rate by claim type:")
print(
    df.groupby("claim_type")[
        "is_fraud_flagged_ground_truth"
    ].mean()
)

print("\nClaim amount statistics:")
print(df.groupby("claim_type")["claim_amount"].describe())
```

Note: This code samples the first 100,000 records, not a statistically random sample. Use reservoir sampling or a properly shuffled sample for final model development.

Also, the dataset card's example loader references a different repository name (FreeSyntheticInsuranceClaims100M). Use the exact repository ID you supplied.

7. Proposed final system architecture

8. Final recommendation

Recommended project direction

Component	Final choice
Primary dataset	FreeInsuranceClaims100M
Secondary dataset	AHRQ SyH-DR
Synthetic data	Custom policy + invoice + settlement generator
Primary ML	CatBoost
Anomaly detection	Isolation Forest
Duplicate detection	RapidFuzz + hashing
Policy engine	Python deterministic rules
Explainability	SHAP
Document AI	OCR + LLM
Backend	FastAPI
Frontend	React
Database	PostgreSQL

One important strategic point: Don't spend the entire development window training a model on 100 million records. Start with a representative sample, establish a baseline, and demonstrate all five required leakage scenarios end-to-end.

Your strongest submission will be one where a user uploads an unseen claim, the application independently evaluates it, calculates the excess payment, displays evidence and allows a human to resolve the finding.

That demonstrates an actual Claims Leakage Detection Agent rather than simply a fraud prediction model.

If you want, I can:

- Define dataset selection criteria list
- Outline research insights and deliverables document
- Suggest next steps integrating FreeInsuranceClaims100M dataset

ASSISTANT

4. Research insights to document

This is important for your submission. You should demonstrate that you understand the problem, not just the technology.

I recommend preparing these research findings.

Insight 1: Fraud detection is not the same as leakage detection

Fraud is intentional deception. Leakage also includes legitimate operational mistakes.

Examples:

- Duplicate invoice accidentally paid twice.
- Incorrect deductible calculation.
- Excluded expense approved by mistake.
- Provider billing exceeds contractual limits.

Research conclusion: A fraud classifier alone cannot solve claims leakage.

Insight 2: Rule-based systems are essential

Some leakage scenarios are deterministic.

For example:

$$\text{Excess} = \max(0, \text{Charged} - \text{Allowed})$$

A machine learning model should not determine whether a 5,000 policy limit was exceeded when the policy explicitly specifies that limit.

Use deterministic calculations for policy compliance.

Insight 3: Class imbalance

Most insurance claims are legitimate.

A model trained on imbalanced data can achieve high accuracy while missing suspicious claims.

Research:

- Fraud-to-legitimate claim ratio.
- Precision and recall.
- PR-AUC.
- False positive rate.
- Cost of missed leakage.

A 99% accuracy model is not necessarily useful.

Insight 4: Cross-claim relationships

A single claim may appear legitimate, but relationships between claims can reveal suspicious patterns.

Example:

- Same invoice submitted under different claim IDs.
- Same provider submitting unusually similar invoices.
- Same incident referenced across multiple claims.
- Multiple claims sharing unusual claimant-provider combinations.

Research graph-based anomaly detection as a potential extension. Research literature also highlights graph-based methods and cost-sensitive classification as promising directions. <Cite refs= {["turn659902search5","turn659902search11"]}/>

Insight 5: Explainability is necessary

Every alert should answer:

1. What was detected?
2. Which rule or model detected it?
3. What evidence supports the finding?
4. How much potential leakage is involved?
5. What should the claims handler investigate?

Insight 6: Missing information should not become a negative finding

If a policy document is missing, the application should report:

INSUFFICIENT_EVIDENCE

It should not assume the claim violates the policy.

---

5. Research deliverables

I would organize your research into these eight deliverables.

- Dataset sources and licenses.
- Dataset schema and field descriptions.
- Missing values and duplicate records.
- Class distribution.
- Fraud vs legitimate claim distribution.
- Feature correlation.
- Dataset limitations and biases.

02. Exploratory Data Analysis (EDA)

- Claim distribution by insurance type.
- Average claim amounts.
- Claim frequency by policyholder tenure.
- Claim amount vs deductible.
- Historical claim frequency.

- Suspicious claim distribution.
- Provider-level billing patterns.

Compare:

Model	Evaluation
Logistic Regression	Baseline
Random Forest	Classification
XGBoost	Classification
CatBoost	Primary model
Isolation Forest	Anomaly detection

Compare precision, recall, F1, PR-AUC, inference latency and training time.

Document:

- Coverage rules.
- Exclusions.
- Deductibles.
- Co-payment.
- Sub-limits.
- Overall benefit limits.
- Expected settlement calculations.

- Exact matching.
- Fuzzy matching.
- Invoice normalization.
- Date and amount similarity.
- Legitimate repeat service identification.

- SHAP feature importance.
- Individual claim explanations.
- Evidence-to-alert mapping.
- Confidence and uncertainty levels.
- Human review workflow.

- Total requested claim amount.
- Calculated excess amount.
- Potential leakage.
- Confirmed leakage.
- Leakage by category.
- Leakage by provider.

- False-positive investigation rate.
- Precision and recall.
- PR-AUC and F1.
- Confusion matrix.
- False-positive analysis.
- Processing throughput.
- Detection latency.
- Model limitations.

6. Dataset exploration code

For 100 million rows, don't load everything into pandas. Start with a streaming sample and use DuckDB or Polars for larger-scale analysis.

```
from datasets import load_dataset
from itertools import islice
import pandas as pd

DATASET = "ziadatalabs/FreeInsuranceClaims100M"

# Stream the dataset instead of loading all 100M records
ds = load_dataset(
    DATASET,
    split="train",
    streaming=True
)

# Take an initial 100,000-row research sample
records = list(islice(ds, 100_000))

df = pd.DataFrame(records)

print("Shape:", df.shape)
print("\nColumns:\n", df.columns.tolist())

print("\nMissing values:\n", df.isnull().sum())

print("\nFraud distribution:")
print(df["is_fraud_flagged_ground_truth"].value_counts(normalize=True))

print("\nClaim distribution:")
print(df["claim_type"].value_counts())

print("\nFraud rate by claim type:")
print(
    df.groupby("claim_type")[
        "is_fraud_flagged_ground_truth"
    ].mean()
)

print("\nClaim amount statistics:")
print(df.groupby("claim_type")["claim_amount"].describe())
```

Note: This code samples the first 100,000 records, not a statistically random sample. Use reservoir sampling or a properly shuffled sample for final model development.

Also, the dataset card's example loader references a different repository name (FreeSyntheticInsuranceClaims100M). Use the exact repository ID you supplied.

7. Proposed final system architecture

8. Final recommendation

Recommended project direction

Component	Final choice
Primary dataset	FreeInsuranceClaims100M
Secondary dataset	AHRQ SyH-DR
Synthetic data	Custom policy + invoice + settlement generator
Primary ML	CatBoost
Anomaly detection	Isolation Forest
Duplicate detection	RapidFuzz + hashing
Policy engine	Python deterministic rules
Explainability	SHAP
Document AI	OCR + LLM
Backend	FastAPI
Frontend	React
Database	PostgreSQL

One important strategic point: Don't spend the entire development window training a model on 100 million records. Start with a representative sample, establish a baseline, and demonstrate all five required leakage scenarios end-to-end.

Your strongest submission will be one where a user uploads an unseen claim, the application independently evaluates it, calculates the excess payment, displays evidence and allows a human to resolve the finding.

That demonstrates an actual Claims Leakage Detection Agent rather than simply a fraud prediction model.